DP (Div. B)

Repaso de máscaras de bits

Hay problemas de DP donde utilizamos números enteros como máscaras de bits para representar conjuntos de elementos cuando el universo es pequeño (al rededor de 20 elementos por lo general). Si en la representación binaria está prendido el bit i, diremos que el i-ésimo elemento está en el conjunto.

Las operaciones de conjuntos se pueden implementar en máscaras de bits utilizando operaciones a nivel de bits. Digamos que $mk1$ es la máscara conteniendo los elementos de A y $mk2$ es la máscara con los elementos de B, entonces:

- Para determinar pertenencia $i \in A$, se escribe `bool(mk1 & (1 << i))`.
- La unión $A \cup B$ es `mk1 | mk2`.
- La intersección $A \cap B$ es `mk1 & mk2`.
- La cardinalidad $|A|$ es `__builtin_popcount(mk1)`.
- Para determinar si $A \subseteq B$ se escribe `bool((mk1 & mk2) == mk1)`.
- Para determinar si $A \supseteq B$ se escribe `bool((mk1 & mk2) == mk2)`.
- Para insertar el elemento i en A se escribe `mk1 |= 1 << i` (ausimiendo que i no está en A). Si no se está seguro si i está en A, se escribe `mk1 |= 1 << i`.
- Para eliminar el elemento i se A se escribe `mk1 |= ~(1 << i)` (ausimiendo que i está en A actualmente). Si no se está seguro si i está en A, se escribe `mk1 = mk1 & ~(1 << i)`.
- El complemento de A asumiendo que el universo tiene k elementos es `((1 << k)-1) ^ mk1`.

A lo largo de las notas de clase estaré utilizando los términos y la notación de conjuntos y sus operaciones equivalentes a nivel de bits de forma intercambeable, con excepción de las implementaciones donde forzosamente utilizaré las operaciones a nivel de bits.

SOS DP

Problema 1

Dado un arreglo $A = [a_0, a_1, \ldots, a_{2^k-1}]$, para cada i tal que $0 \leq i < 2^k$, hallar la suma de las submáscaras de i. Dicho de otra forma, queremos obtener un arreglo $A' = [a'_0, a'_1, \ldots, a'_{2^k-1}]$, tal que:

$$a'_i = \sum_{s \subseteq i} a_s$$

Ejemplo:

Sea $A = [4, 7, 1, 0, 2, 3, 1, 5]$. Entonces los elementos de A' serían:

$a'_0 = a_0 = 4$
$a'_1 = a_0 + a_1 = 4 + 7 = 11$
$a'_2 = a_0 + a_2 = 4 + 1 = 5$
$a'_3 = a_0 + a_1 + a_2 + a_3 = 4 + 7 + 1 + 0 = 12$
$a'_4 = a_0 + a_4 = 4 + 2 = 6$
$a'_5 = a_0 + a_1 + a_4 + a_5 = 4 + 7 + 2 + 3 = 16$
$a'_6 = a_0 + a_2 + a_4 + a_6 = 4 + 1 + 2 + 1 = 8$
$a'_7 = a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 = 4 + 7 + 1 + 0 + 2 + 3 + 1 + 5 = 23$

La respuesta debería ser el arreglo $A' = [4, 11, 5, 12, 6, 16, 8, 23]$.

Este es el problema que resuelve la DP conocida como Sum Over Subsets (SOS). A continuación veremos algunas soluciones.

Solución $O(4^k)$

La solución más bruta es: para cada máscara mk, iterar sobre todas las máscaras y sumar aquellas que sean su submáscara:

```
for (int mk = 0; mk < (1 << k); mk++) {
    Ap[mk] = 0;
    for (int s = 0; s < (1 << k); s++)
        if ((mk & s) == s)
            Ap[mk] += A[s];
}
```

Solución $O(3^k)$

En la solución anterior iteramos sobre todos los elementos de A para obtener b_{mk}. Pero, ¿qué tanto podemos mejorar la solución si sólo iteramos por las submáscaras de mk? Esto lo podemos hacer de la siguiente forma:

```
for (int mk = 0; mk < (1 << k); mk++) {
    Ap[mk] = 0;
    for (int s = mk;; s = (s - 1) & mk) {
        Ap[mk] += A[s];
        if (!s)
            break;
    }
}
```

Veamos un ejemplo de como se recorren las submáscaras de $mk = 154$:

▼ Ejemplo

Operacion	Máscara	Valor númerico	# submáscara
$s = mk$	$\underline{1}00\underline{11}0\underline{1}0_2$	154	15
$s - 1$	$\underline{1}00\underline{11}0\underline{0}1_2$	153	
$s = (s-1)\&mk$	$\underline{1}00\underline{11}0\underline{0}0_2$	152	14
$s - 1$	$\underline{1}00\underline{1}0\underline{1}1_2$	151	
$s = (s-1)\&mk$	$\underline{1}00\underline{1}0\underline{0}1_2$	146	13
$s - 1$	$\underline{1}00\underline{1}0\underline{0}1_2$	145	
$s = (s-1)\&mk$	$\underline{1}00\underline{1}0\underline{0}0_2$	144	12
$s - 1$	$\underline{1}00\underline{0}1\underline{1}1_2$	143	
$s = (s-1)\&mk$	$\underline{1}00\underline{0}1\underline{0}1_2$	138	11
$s - 1$	$\underline{1}00\underline{0}1\underline{0}1_2$	137	
$s = (s-1)\&mk$	$\underline{1}00\underline{0}1\underline{0}0_2$	136	10
$s - 1$	$\underline{1}000\underline{1}1_2$	135	
$s = (s-1)\&mk$	$\underline{1}000\underline{0}1_2$	130	9
$s - 1$	$\underline{1}000\underline{0}1_2$	129	
$s = (s-1)\&mk$	$\underline{1}000\underline{0}0_2$	128	8
$s - 1$	$\underline{0}111\underline{1}1_2$	127	
$s = (s-1)\&mk$	$\underline{0}00\underline{11}0\underline{1}0_2$	26	7
$s - 1$	$\underline{0}00\underline{11}0\underline{0}1_2$	25	
$s = (s-1)\&mk$	$\underline{0}00\underline{11}0\underline{0}0_2$	24	6
$s - 1$	$\underline{0}00\underline{1}0\underline{1}1_2$	23	
$s = (s-1)\&mk$	$\underline{0}00\underline{1}0\underline{0}1_2$	18	5
$s - 1$	$\underline{0}00\underline{1}0\underline{0}1_2$	17	
$s = (s-1)\&mk$	$\underline{0}00\underline{1}0\underline{0}0_2$	16	4
$s - 1$	$\underline{0}000\underline{1}1_2$	15	
$s = (s-1)\&mk$	$\underline{0}000\underline{1}0\underline{1}0_2$	10	3
$s - 1$	$\underline{0}000\underline{1}0\underline{0}1_2$	9	

Operacion	Máscara	Valor número	# submáscara
$s = (s-1)\&mk$	$\underline{0}0\underline{0}0\underline{1}\underline{0}\underline{0}\underline{0}_2$	8	2
$s-1$	$\underline{0}0\underline{0}0\underline{0}\underline{1}\underline{1}\underline{1}_2$	7	
$s = (s-1)\&mk$	$\underline{0}0\underline{0}0\underline{0}\underline{0}\underline{1}\underline{0}_2$	2	1
$s-1$	$\underline{0}0\underline{0}0\underline{0}\underline{0}\underline{0}\underline{1}_2$	1	
$s = (s-1)\&mk$	$\underline{0}0\underline{0}0\underline{0}\underline{0}\underline{0}\underline{0}_2$	0	0

Complejidad

La complejidad de esta solución se puede obtener si analizamos sobre qué pares (mk, s) iteran los dos ciclos `for` anidados. Hagamos el análisis bit por bit:

- Para los bits que sean 0 en mk, sólo vamos a iterar sobre submáscaras s que sean 0 en esos bits.
- Para los bits que sean 1 en mk, podemos iterar sobre submáscaras s que tengan 0 o 1 en esos bits.

Entonces para cada bit, hay 3 combinaciones posibles:

1. Que sea 0 tanto en mk como en s.
2. Que sea 1 en mk y 0 en s.
3. Que sea 1 tanto en mk como en s.

Para cada bit tenemos 3 combinaciones, entonces iteramos sobre 3^k parejas (mk, s) , por lo que la complejidad es $O(3^k)$.

Solución $O(k2^k)$

Hasta el momento no hemos usado la técnica de DP para atacar el problema. ¿Qué función de DP podríamos definir para encontrar los elementos de A'?

Una forma de encontrar soluciones DP es pensar en una función recursiva de fuerza bruta, y después memorizar sobre los argumentos de la función. Una posible función que cumple con dichas características para este problema es $f(mask, i)$, la cual regresa la suma sobre todas las submáscaras de $mask$ que tienen los primeros i bits iguales a los de $mask$ (y los demás bits pueden o no ser iguales).

La función quedaría definida como:

$$f(mask, i) = \begin{cases} a[mask] & \text{si } i = k \\ f(mask, i+1) + f(mask \text{ XOR } 2^i, i+1) & \text{si } i \in mask \\ f(mask, i+1) & \text{en caso contrario} \end{cases}$$

El valor de a'_i es $f(mask, 0)$.

$mask$ tiene 2^k valores distinos, i tiene $k+1$ valores distintos, por lo tanto hay $(k+1)2^k)$ estados distintos. Las transiciones son en tiempo constante, por lo tanto la solución con DP queda en $O(k2^k)$.

La DP recursiva se puede programar facilmente teniendo la función $f(mask, i)$. La versión iterativa, quedaría de la siguiente forma:

```
vector<int> SOS_DP(vector<int> A, int k) {
    vector<vector<int>> dp(k + 1, vector<int>(1 << k));

    for (int mk = 0; mk < (1 << k); mk++)
        dp[k][mk] = A[mk];

    for (int i = k - 1; i >= 0; i--)
        for (int mk = 0; mk < (1 << k); mk++) {
            if (mk & (1 << i))
                dp[i][mk] = dp[i + 1][mk] + dp[i + 1][mk - (1 << i)];
            else
                dp[i][mk] = dp[i + 1][mk];
        }

    vector<int> Ap = dp[0];
    return Ap;
}
```

Los índices en la implementación de `dp` están invertidos con respecto a $f(mask, i)$. Esto es por optimizar el caché del programa y facilitar el obtener el arreglo `Ap` antes del return.

Solución con memoria lineal

La solución anterior es la mejor solución en tiempo. Sin embargo, utiliza $O(k2^k)$ de memoria. Se puede mejorar la DP para que utilice memoria lineal.

Notemos que $f(mask, i)$ sólo depende de estados con $i-1$ y con $mask$ que sea menor o igual a $mask$, por lo que podemos implementar la DP recursiva de la siguiente forma:

```
vector<int> SOS_DP(vector<int> A, int k) {
    vector<int> Ap = A;
    for (int i = k - 1; i >= 0; i--)
        for (int mk = 0; mk < (1 << k); mk++)
                if (mk & (1 << i))
                        Ap[mk] += Ap[mk - (1 << i)];
    return Ap;
}
```

La complejidad en tiempo también es $O(k2^k)$, pero la memoria ahora es lineal.

Inversa de la SOS DP

Ahora queremos resolver el problema inverso.

> ℹ️ **Problema 2**
> Dado el arreglo $A' = [a'_0, a'_1, \ldots, a'_{2^k-1}]$, donde cada elemento cumple la siguiente igualdad:
>
> $$a'_i = \sum_{s \subseteq i} a_s$$
>
> Encontrar el arreglo A.
>
> **Ejemplo:**
> Sea $A' = [4, 11, 5, 12, 6, 16, 8, 23]$. Este es el arreglo A' que se obtiene en el ejemplo del problema 1, así que la respuesta es el arreglo $A = [4, 7, 1, 0, 2, 3, 1, 5]$.

Solución $O(4^k)$ o $O(3^k)$

Obtener el valor de a_0 es fácil, pues $a'_0 = a_0$. Ahora supóngamos que ya tenemos el valor de a_i para todos los valores de $i < mask$, ¿cómo podemos hallar el valor de a_{mask}?

Recordemos que:

$$a'_i = \sum_{s \subseteq i} a_s = a_i + \sum_{s \subsetneq i} a_s$$

Despejando a_i tenemos:

$$a_i = a'_i - \sum_{s \subsetneq i} a_s$$

La última suma itera sobre valores s que son menores a i. Así podemos hallar a_i teniendo los valores de índices menores.

Para implementar la solución, podemos hacer la bruta en $O(4^k)$ o iterar sólo sobre todas las submáscaras para tener una solución en $O(3^k)$. La implementación de esta última sería:

```
vector<int> SOS_DP_inv (vector<int> Ap, int k) {
    vector<int> A(1 << k);

    for (int mk = 0; mk < (1 << k); mk++) {
            A[mk] = Ap[mk];
            for (int s = mk; s != 0;) {
                s = (s - 1) & mk;
                A[mk] -= A[s];
            }
        }

    return A;
}
```

Solución $O(k2^k)$

Ahora veamos cómo invertir la DP usada anteriormente. Recordemos que $f(mask, k) = a_{mask}$ y que $f(mask, 0) = a'_{mask}$. Podemos recuperar fácilmente el valor de $f(mask, 0)$ y queremos obtener el valor de $f(mask, k)$. ¿Cómo recuperamos $f(mask, k)$? Vamos a reconstruir todos los valores de la DP a partir de $f(mask, 0)$.

Esto lo vamos a hacer desde $i = 0$ hasta $i = k$. Si el bit i no está prendido en $mask$, observemos que simplemente $f(mask, i + 1) = f(mask, i)$. Y cuando está prendido teníamos $f(mask, i) = f(mask, i + 1) - f(mask - 2^i, i + 1)$; como queremos hallar $f(mask, i + 1)$, lo despejamos en la igualdad anterior: $f(mask, i + 1) = f(mask, i) - f(mask - 2^i, i + 1)$. Observemos que $f(mask, i + 1)$ depende de estados con i menor o de estados con $mask$ menor, por lo que si procesamos los estados con i de menor a mayor y $mask$ de menor a mayor, podemos recuperar todos los estados de la DP.

La implementación quedaría así:

```
vector<int> SOS_DP_inv (vector<int> Ap, int k) {
    vector<vector<int>> dp(k + 1, vector<int>(1 << k));
    dp[0] = Ap;

    for (int i = 0; i < k; i++)
        for (int mk = 0; mk < (1 << k); mk++)
            if (mk & (1 << i))
                dp[i + 1][mk] = dp[i][mk] - dp[i + 1][mk - (1 << i)];
            else
                dp[i + 1][mk] = dp[i][mk];

    vector<int> A = dp[k];
    return A;
}
```

Solución con memoria constante

Podemos ahorrar memoria y utilizar una cantidad lineal. Esto se logra con la misma idea con la que se optimizó la memoria de la SOS DP:

```
vector<int> SOS_DP_inv (vector<int> Ap, int k) {
    vector<int> A = Ap;
    for (int i = 0; i < k; i++)
        for (int mk = 0; mk < (1 << k); mk++)
            if (mk & (1 << i))
                A[mk] -= A[mk - (1 << i)];
    return A;
}
```

SOS DP para superconjuntos

Problema 3

Dado un arreglo $A = [a_0, a_1, \ldots, a_{2^k-1}]$, para cada i tal que $0 \leq i < 2^k$, hallar la suma de las supermáscaras de i. Dicho de otra forma, queremos obtener un arreglo $A' = [a'_0, a'_1, \ldots, a'_{2^k-1}]$, tal que:

$$a'_i = \sum_{s \supseteq i} a_s$$

Ejemplo:

Sea $A = [4, 7, 1, 0, 2, 3, 1, 5]$. Entonces los elementos de A' serían:

$a'_0 = a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 = 4 + 7 + 1 + 0 + 2 + 3 + 1 + 5 = 23$
$a'_1 = a_1 + a_3 + a_5 + a_7 = 7 + 0 + 3 + 5 = 15$
$a'_2 = a_2 + a_3 + a_6 + a_7 = 1 + 0 + 1 + 5 = 7$
$a'_3 = a_3 + a_7 = 0 + 5 = 5$
$a'_4 = a_4 + a_5 + a_6 + a_7 = 2 + 3 + 1 + 5 = 11$
$a'_5 = a_5 + a_7 = 3 + 5 = 8$
$a'_6 = a_6 + a_7 = 1 + 5 = 6$
$a'_7 = a_7 = 5 = 5$

La respuesta debería ser el arreglo $A' = [23, 15, 7, 5, 11, 8, 6, 5]$.

Las soluciones $O(4^k)$ y $O(3^k)$ son muy parecidas a las de SOS DP para submáscaras, entonces nos las vamos a saltar.

Solución $O(k2^k)$

Utilicemos una solución DP parecida a la que usamos en la SOS DP de subconjuntos. Definamos a $f(mask, i)$ como la suma de todas las supermáscaras de $mask$ que tienen los primers i bits idénticos a $mask$. La función $f(mask, i)$ se puede definir como:

$$f(mask, i) = \begin{cases} a[mask] & \text{si } i = k \\ f(mask, i+1) + f(mask + 2^i, i+1) & \text{si } i \notin mask \\ f(mask, i+1) & \text{en caso contrario} \end{cases}$$

La implementación es muy parecida la DP anterior:

```
vector<int> SUPER_SOS_DP(vector<int> A, int k) {
    vector<vector<int>> dp(k + 1, vector<int>(1 << k));

    for (int mk = 0; mk < (1 << k); mk++)
        dp[k][mk] = A[mk];

    for (int i = k - 1; i >= 0; i--)
        for (int mk = 0; mk < (1 << k); mk++) {
            if (~mk & (1 << i))
                dp[i][mk] = dp[i + 1][mk] + dp[i + 1][mk + (1 << i)];
            else
                dp[i][mk] = dp[i + 1][mk];
        }

    vector<int> Ap = dp[0];
    return Ap;
}
```

El análisis de complejidad es similar: tiempo y memoria $O(k2^k)$

Solución con ahorro de memoria

Esta DP también depende de los estados con $i-1$, pero ahora depende de máscaras mayores o iguales a $mask$, por lo que la implementación sería:

```
vector<int> SUPER_SOS_DP(vector<int> A, int k) {
    vector<int> Ap = A;
    for (int i = k - 1; i >= 0; i--)
        for (int mk = 0; mk < (1 << k); mk++)
                if (~mk & (1 << i))
                        Ap[mk] += Ap[mk + (1 << i)];
    return Ap;
}
```

Inversa de la SOS DP para superconjuntos

ℹ️ **Problema 4**
Dado el arreglo $A' = [a'_0, a'_1, \ldots, a'_{2^k-1}]$, donde cada elemento cumple la siguiente igualdad:

$$a'_i = \sum_{s \supseteq i} a_s$$

Encontrar el arreglo A.

Ejemplo:
Sea $A' = [23, 15, 7, 5, 11, 8, 6, 5]$. Este es el arreglo A' que se obtiene en el ejemplo del problema 3, así que la respuesta es el arreglo $A = [4, 7, 1, 0, 2, 3, 1, 5]$.

Solución $O(4^k)$ o $O(3^k)$

Ahora tenemos que $a_{2^k-1} = a'_{2^k-1}$, ya que $2^k - 1$ es la única supermáscara de $2^k - 1$. Para hallar todas las a_i, observemos que

$$a_i = a'_i - \sum_{s \supsetneq i} a_s$$

Los valores de s por los que itera la suma son mayores a i. Juntando las observaciones hechas, llegamos a la conclusión de que podemos hallar los valores de a_i desde los índices más grandes a los más pequeños.

Iterar sobre las supermáscaras de un número es más díficil que iterar sobre las submáscaras. Por lo que ahora iteramos sobre las máscaras de mayor a menor, y cada máscara resta su valor de sus submáscaras:

```
vector<int> SUPER_SOS_DP_inv (vector<int> Ap, int k) {
    vector<int> A(1 << k);

    for (int mk = (1 << k) - 1; mk >= 0; mk--) {
            A[mk] += Ap[mk];
            for (int s = mk; s != 0;) {
                s = (s - 1) & mk;
                A[s] -= A[mk];
            }
        }

    return A;
}
```

La complejidad es $O(3^k)$.

Solución $O(k2^k)$

Usemos las siguientes observaciones sobre la función $f(mask, i)$ utilizada en SOS para suberconjuntos para llegar a una solución parecida a la inversa en SOS DP de subconjuntos:

- $f(mask, 0) = a'_{mask}$
- $f(mask, k) = a_{mask}$
- Si el bit i está prendido en $mask$, entonces $f(mask, i + 1) = f(mask, i)$.
- Si el bit i no está prendido en $mask$, entonces $f(mask, i) = f(mask, i + 1) + f(mask - 2^i, i + 1)$.

Un estado depende de estados con i menor o con $mask$ mayor, así que la implementación se puede hacer de la siguiente forma:

```
vector<int> SUPER_SOS_DP_inv (vector<int> Ap, int k) {
    vector<vector<int>> dp(k + 1, vector<int>(1 << k));
    dp[0] = Ap;

    for (int i = 0; i < k; i++)
        for (int mk = (1 << k) - 1; mk >= 0; mk--)
            if (~mk & (1 << i))
                dp[i + 1][mk] = dp[i][mk] - dp[i + 1][mk + (1 << i)];
            else
                dp[i + 1][mk] = dp[i][mk];

    vector<int> A = dp[k];
    return A;
}
```

Solución con memoria constante

Para esta inversa también podemos utilizar memoria lineal:

```
vector<int> SUPER_SOS_DP_inv (vector<int> Ap, int k) {
    vector<int> A = Ap;
    for (int i = 0; i < k; i++)
        for (int mk = (1 << k) - 1; mk >= 0; mk--)
            if (~mk & (1 << i))
                A[mk] -= A[mk + (1 << i)];
    return A;
}
```

SOS DP para el complemento

Problema 5

Dado un arreglo $A = [a_0, a_1, \ldots, a_{2^k-1}]$, para cada i tal que $0 \leq i < 2^k$, hallar la suma de las posiciones cuyo índice no tiene bits en común con i. Dicho de otra forma, queremos obtener un arreglo $A' = [a'_0, a'_1, \ldots, a'_{2^k-1}]$, tal que:

$$a'_i = \sum_{s \cap i = 0} a_s$$

Ejemplo:

Sea $A = [4, 7, 1, 0, 2, 3, 1, 5]$. Entonces los elementos de A' serían:

$a'_0 = a_0 + a_1 + a_2 + a_3 + a_4 + a_5 + a_6 + a_7 = 4 + 7 + 1 + 0 + 2 + 3 + 1 + 5 = 23$
$a'_1 = a_0 + a_2 + a_4 + a_6 = 4 + 1 + 2 + 1 = 8$
$a'_2 = a_0 + a_1 + a_4 + a_5 = 4 + 7 + 2 + 3 = 16$
$a'_3 = a_0 + a_4 = 4 + 2 = 6$
$a'_4 = a_0 + a_1 + a_2 + a_3 = 4 + 7 + 1 + 0 = 12$
$a'_5 = a_0 + a_2 = 4 + 1 = 5$
$a'_6 = a_0 + a_1 = 4 + 7 = 11$
$a'_7 = a_0 = 4$

La respuesta debería ser el arreglo $A' = [23, 8, 16, 6, 12, 5, 11, 4]$.

La condición de que i y s no compartan bits, es que todos los bits que tenga prendidos s los tenga apagados i, y los que tenga prendidos i los renga apagados s. Esto es equivalente a decir que s sea una submáscara del complemento de i. Por lo que la suma que quieremos hallar es:

$$a'_i = \sum_{s \subseteq i^C} a_s$$

Para hallar A', podemos hacer SOS DP en A, y asignar a_i al indíce que sea complemento, es decir, $i \oplus ((1 << k) - 1)$.

Aplicación: convoluciones a nivel de bits

Convolución OR

> ℹ️ **Problema 6**
> Dado dos arreglos de tamaño 2^k $A = [a_0, a_1, \ldots, a_{2^k-1}]$ y $B = [b_0, b_1, \ldots, b_{2^k-1}]$, queremos calcular otro arreglo $C = [c_0, c_1, \ldots, c_{2^k-1}]$, donde el elemento c_i es la suma de los productos de parejas de elementos de A y B, tal que el OR a nivel de bits de sus índices es igual a i. Formalmente:
>
> $$c_i = \sum_{j|l=i} a_j \cdot b_l$$

Solución bruta

La solución bruta sería iterar sobre todas las parejas de elementos de A y B:

```
for (int i = 0; i < (1 << k); i++)
    for (int j = 0; j < (1 << k); j++)
        C[i|j] += A[i] * B[j];
```

Solución con SOS DP

Digamos que obtenemos la SOS DP de A y B, a las que llamaremos A' y B', es decir, $A' = sos_dp(A, k)$ y $B' = sos_dp(B, k)$.

Ahora calculemos el producto punto a punto de A' y B' al cual llamaremos C', es decir, $C' = [a'_0 b'_0, a'_1 b'_1, \ldots, a'_{2^k-1} b'_{2^k-1}]$.

¿Qué significado tiene c'_i? Tenemos $c'_i = a'_i \cdot b'_i$, donde a'_i es la suma de todas las submáscaras de i en A, y b'_i es la suma de todas las submáscaras de i en B, por lo tanto, c'_i es la suma de los productos de parejas a_j y b_l, donde tanto j como l son submáscaras de i; como ambas son submáscaras de i, entonces su OR ($j|l$), también es submáscara de i. Esto quiere decir que c'_i es la suma de todas las c_j donde j es submáscara de i.

▼ Demostración

$$
\begin{aligned}
c'_i &= a'_i b'_i \\
&= (\sum_{j \subseteq i} a_j) \cdot (\sum_{l \subseteq i} b_l) \\
&= \sum_{j,l \subseteq i} a_j b_l \\
&= \sum_{j|l \subseteq i} a_j b_l \text{ (ya que } j, l \subseteq i \text{ es equivalente a } j|l \subseteq i) \\
&= \sum_{s \subseteq i} \sum_{j|l=s} a_j b_l \\
&= \sum_{s \subseteq i} c_s
\end{aligned}
$$

Después de entender el significado que tiene c'_i, podemos ver que sólo basta hacerle la SOS DP inversa para obtener c_i. El código sería:

```
Ap = SOS_DP(A, k);
Bp = SOS_DP(B, k);

for (int i = 0; i < (1 << k); i++)
    Cp[i] = Ap[i] * Bp[i];

C = SOS_DP_inv(Cp, k);
```

Complejidad: $O(k2^k)$

(https://hackmd.io/@BUL97mR / DP (Div. B) 1

Convolución AND

> ℹ️ **Problema 7**
> Dado dos arreglos de tamaño 2^k $A = [a_0, a_1, \ldots, a_{2^k-1}]$ y $B = [b_0, b_1, \ldots, b_{2^k-1}]$, queremos calcular otro arreglo $C = [c_0, c_1, \ldots, c_{2^k-1}]$, donde el elemento c_i es la suma de los productos de parejas de elementos de A y B, tal que el AND a nivel de bits de sus índices es igual a i. Formalmente:
>
> $$c_i = \sum_{j \& l = i} a_j \cdot b_l$$

Solución con SOS DP

Ahora aprovechemos que la condición $j \& l \supseteq i$ es equivalente a que $j \& l$ sea supermáscara de i. Podemos modificar la solución anterior y utilizar la SOS DP para superconjuntos.

▼ Demostración

$$
\begin{aligned}
c'_i &= a'_i b'_i \\
&= (\sum_{j \supseteq i} a_j) \cdot (\sum_{l \supseteq i} b_l) \\
&= \sum_{j,l \supseteq i} a_j b_l \\
&= \sum_{j \& l \supseteq i} a_j b_l \ (\text{ya que } j, l \supseteq i \text{ es equivalente a } j \& l \supseteq i) \\
&= \sum_{s \supseteq i} \sum_{j \& l = s} a_j b_l \\
&= \sum_{s \supseteq i} c_s
\end{aligned}
$$

```
Ap = SUPER_SOS_DP(A, k);
Bp = SUPER_SOS_DP(B, k);

for (int i = 0; i < (1 << k); i++)
    Cp[i] = Ap[i] * Bp[i];

C = SUPER_SOS_DP_inv(Cp, k);
```

Complejidad: $O(k2^k)$

Lectura recomendada

Explicaciones alternativas de SOS DP

- https://codeforces.com/blog/entry/45223 (https://codeforces.com/blog/entry/45223)
- https://usaco.guide/adv/dp-sos (https://usaco.guide/adv/dp-sos)
- https://codeforces.com/blog/entry/105247 (https://codeforces.com/blog/entry/105247)

Convolución OR y otras convoluciones

- https://codeforces.com/blog/entry/115438 (https://codeforces.com/blog/entry/115438) (recomendado leer también los comentarios para ver cómo se puede generalizar la idea a más convoluciones)

Convolución XOR explicado con FFT

- https://hackmd.io/@8iUL97mRS2mgfTEJDy5xmA/H1EYLooPC (https://hackmd.io/@8iUL97mRS2mgfTEJDy5xmA/H1EYLooPC)

Problemas

- https://cses.fi/problemset/task/1654 (https://cses.fi/problemset/task/1654)
- https://codeforces.com/contest/165/problem/E
  (https://codeforces.com/contest/165/problem/E)
- https://codeforces.com/contest/383/problem/E
  (https://codeforces.com/contest/383/problem/E)
- https://usaco.org/index.php?page=viewproblem2&cpid=1309
  (https://usaco.org/index.php?page=viewproblem2&cpid=1309)
- https://codeforces.com/contest/1208/problem/F
  (https://codeforces.com/contest/1208/problem/F)